rative summary

Computer Organization and Architecture
(Code: CS256)

Lecture-25: Cache Performance

Course Faculty

Dr. B. R. Bhowmik

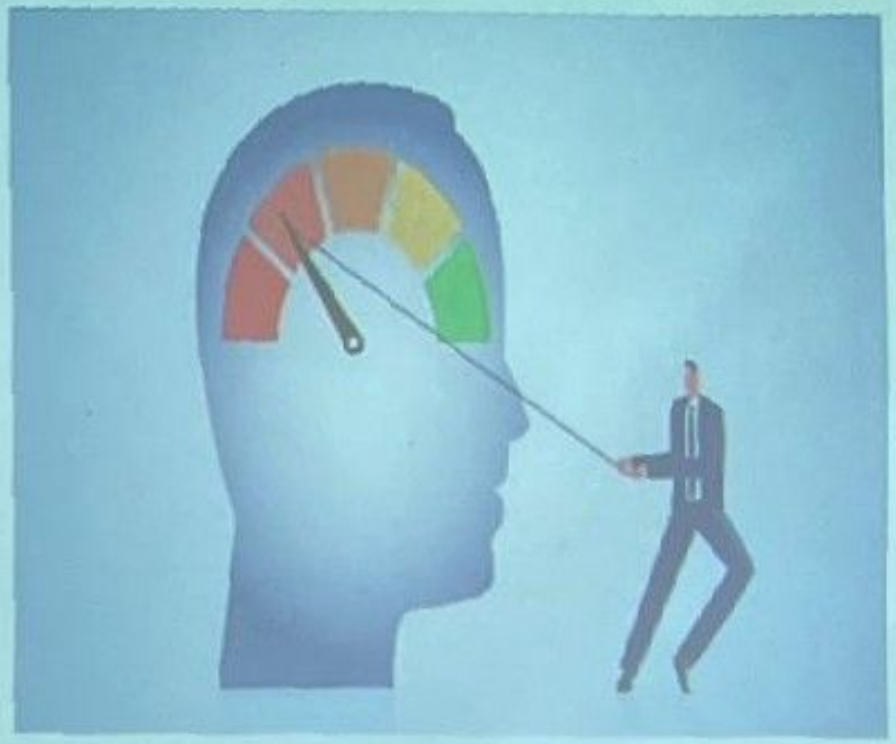
Assistant Professor

Dept. of CSE

Website: https://sites.google.com/view/biswajitrbhowmik
https://sites.google.com/view/vicayana-gaveshana

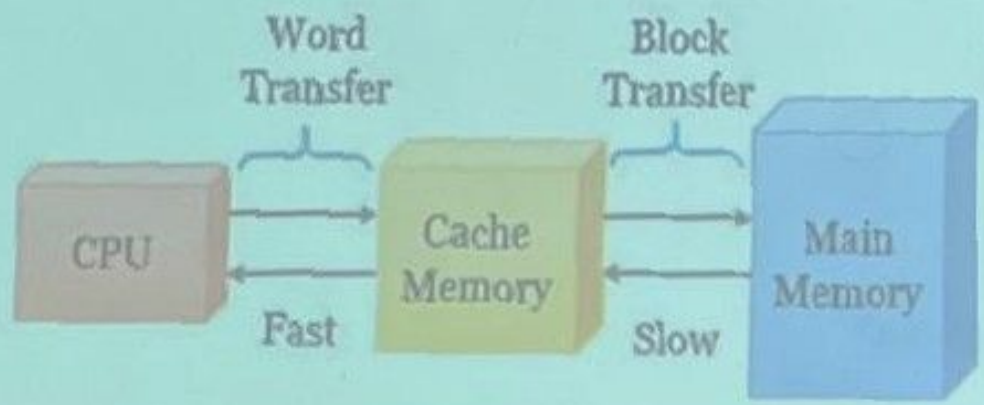
Course Email: sangsangstha@gmail.com

Performance

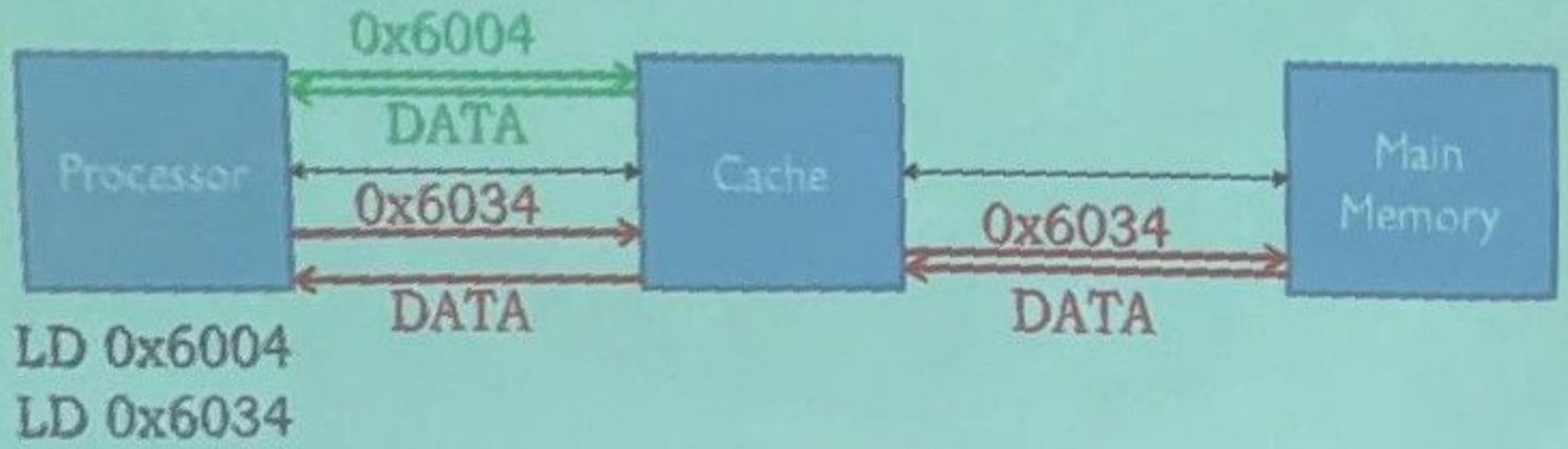


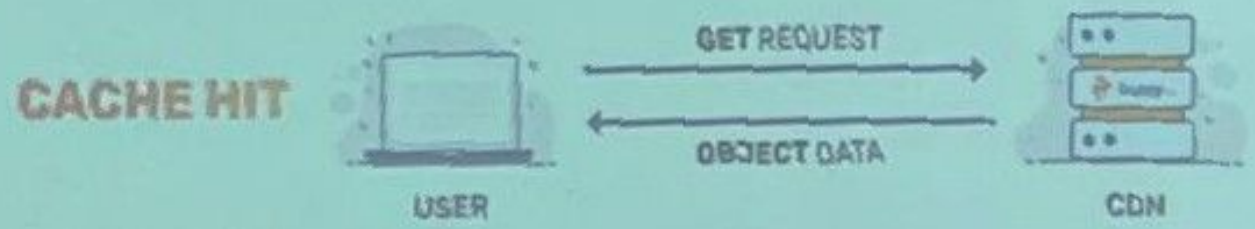
Cache Performance

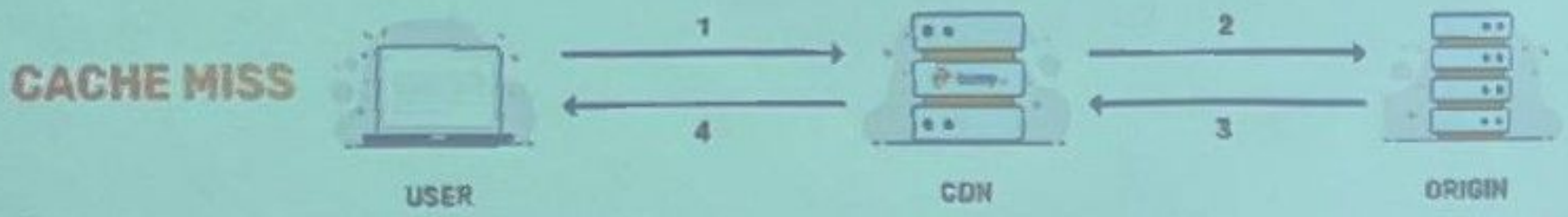
- The performance of the cache memory is measured in terms of a quantity called ***Hit Ratio***.
- ***Cache Hit:*** When the CPU refers to the memory and reveals the word in the cache, it's far stated that a **hit** has successfully occurred.
- ***Cache Miss:*** If the word is not discovered in the cache, then the CPU refers to the main memory for the favored word and it is referred to as a ***miss*** to cache.



Cache Performance





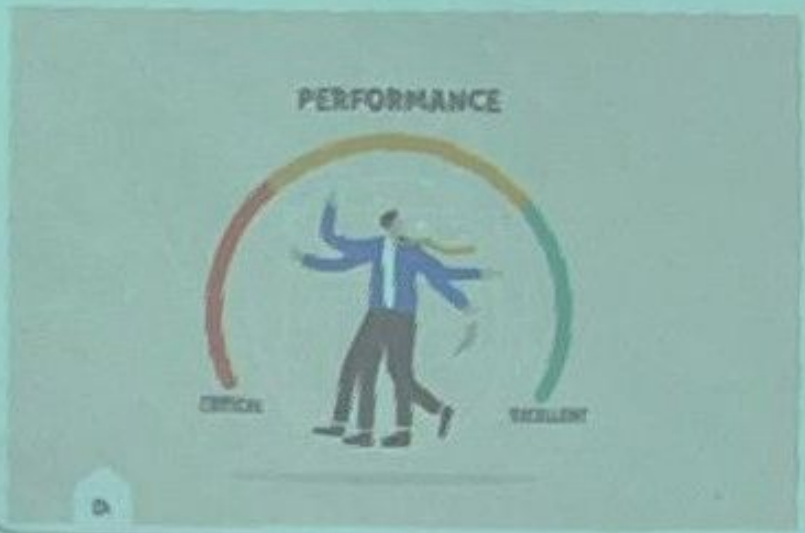


Hit Ratio

- The **hit ratio H,** is nothing but a probability of getting hits out of some number of memory references made by the CPU.
- So its range is $0 \leq H \leq 1$.

Hit Ratio = *Number of Hits / Total CPU references to memory*

= *Number of hits / (Number of Hits + Number of Misses)*

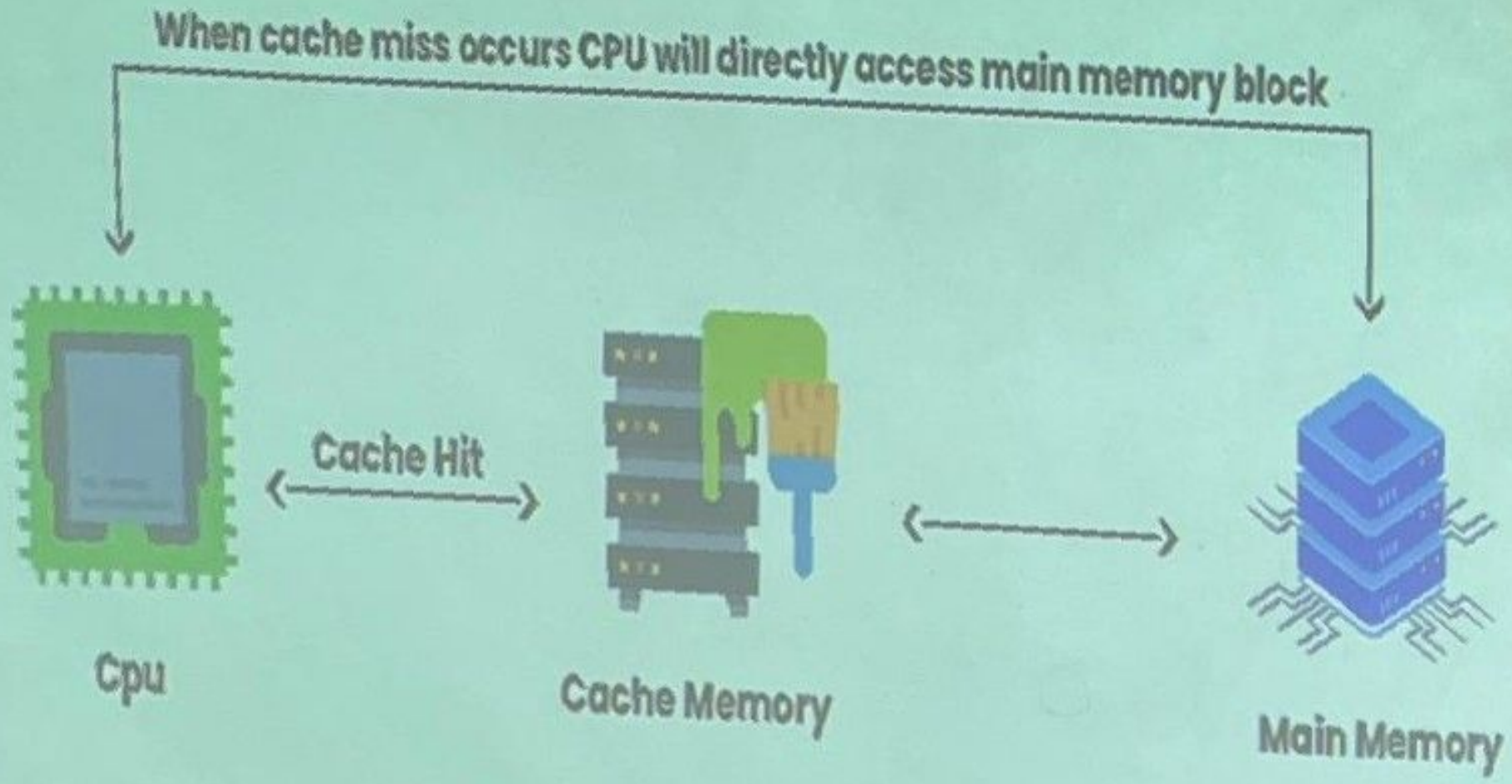


Hit Ratio

$$\frac{\text{Cache Hits}}{\text{Total Cache Request}} = \text{CACHE HIT RATIO}$$

(Cache Hits + Cache Misses)

AMAT or EMAT

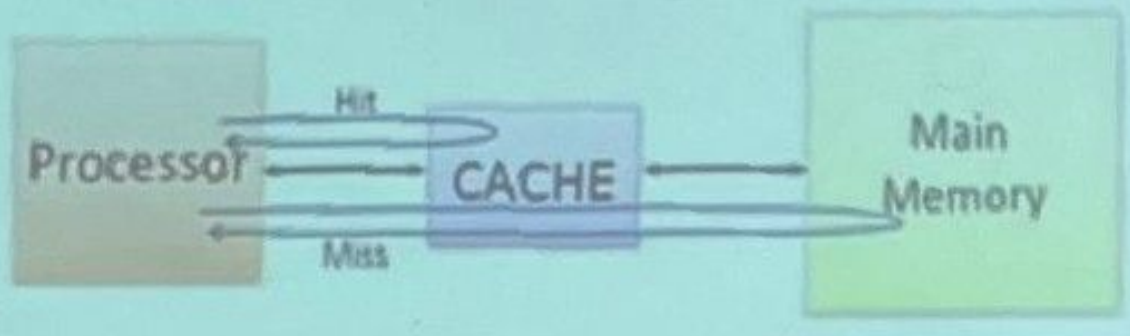

When cache miss occurs CPU will directly access main memory block
Cache Hit
Cpu
Cache Memory
Main Memory

AMAT

- In order to look at the performance of cache memories, we need to look at the *average memory access time* (AMAT) and the factors that will affect it.

$$\text{AMAT} = \text{Hit Time} + \text{Miss Time}$$
$$= \text{Hit Time} + \text{Miss Rate} \times \text{Miss Penalty}$$

- **Miss Rate :** the fraction of accesses that are not in the cache
  - (1-H).
- **Miss Penalty :** the additional clock cycles to service the miss
  - the extra time needed to carry the favored information into cache from main memory in case of miss in cache.



Cache Performance

- The performance of a single-level cache system for a read operation can be characterized by the following equation:

$$Ta = HTc + (1 - H) \times (Tm + Tc)$$
$$= Tc + (1 - H) \times Tm$$

- Here

Ta = the average memory access time (AMAT),

Tc = the cache access time,

Tm = the memory access time (e.g., memory to processor register),

H = the hit ratio.

Cache Performance Improvement

- Three basic ways to improve cache performance:
  - Reducing hit time
  - Reducing miss rate
  - Reducing miss penalty
  - *Reducing miss penalty X rate via parallelism*
  - *Advanced optimization techniques*

- Note that by Amdahl's Law, there will be diminishing returns from reducing only hit time or amortized miss penalty by itself, instead of both together.

Hit Time Reduction Techniques

- Small and simple cache
- Avoid address translation in indexing cache, i.e., address translation
- Pipelined cache access
- Trace caches

Miss Rate Reduction Techniques

- Larger block size
- Increase cache size
- Higher associativity
- Way prediction
- Pseudo-associative caches
- Compiler optimizations

Miss Penalty Reduction Techniques

- Multilevel cache
- Critical word first and early restart
- Priority to read miss
- Merging write buffer
- Victim cache

Miss Penalty X Rate Reduction Techniques

- Non-blocking cache
- Hardware prefetching
- Compiler-controlled prefetching

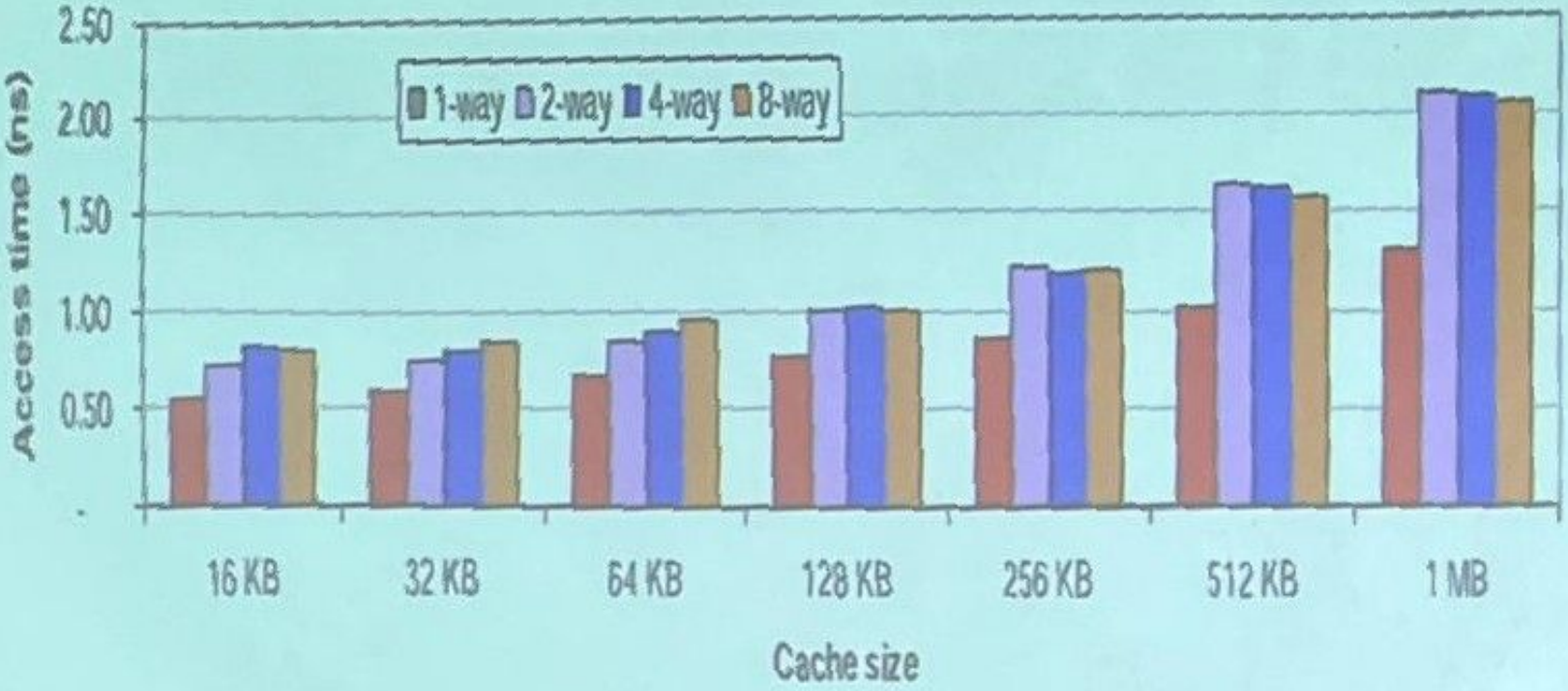
Advanced Optimization Techniques

- Pipelined caches
- Multi-banked caches
- Non-blocking caches

Small and Simple Caches

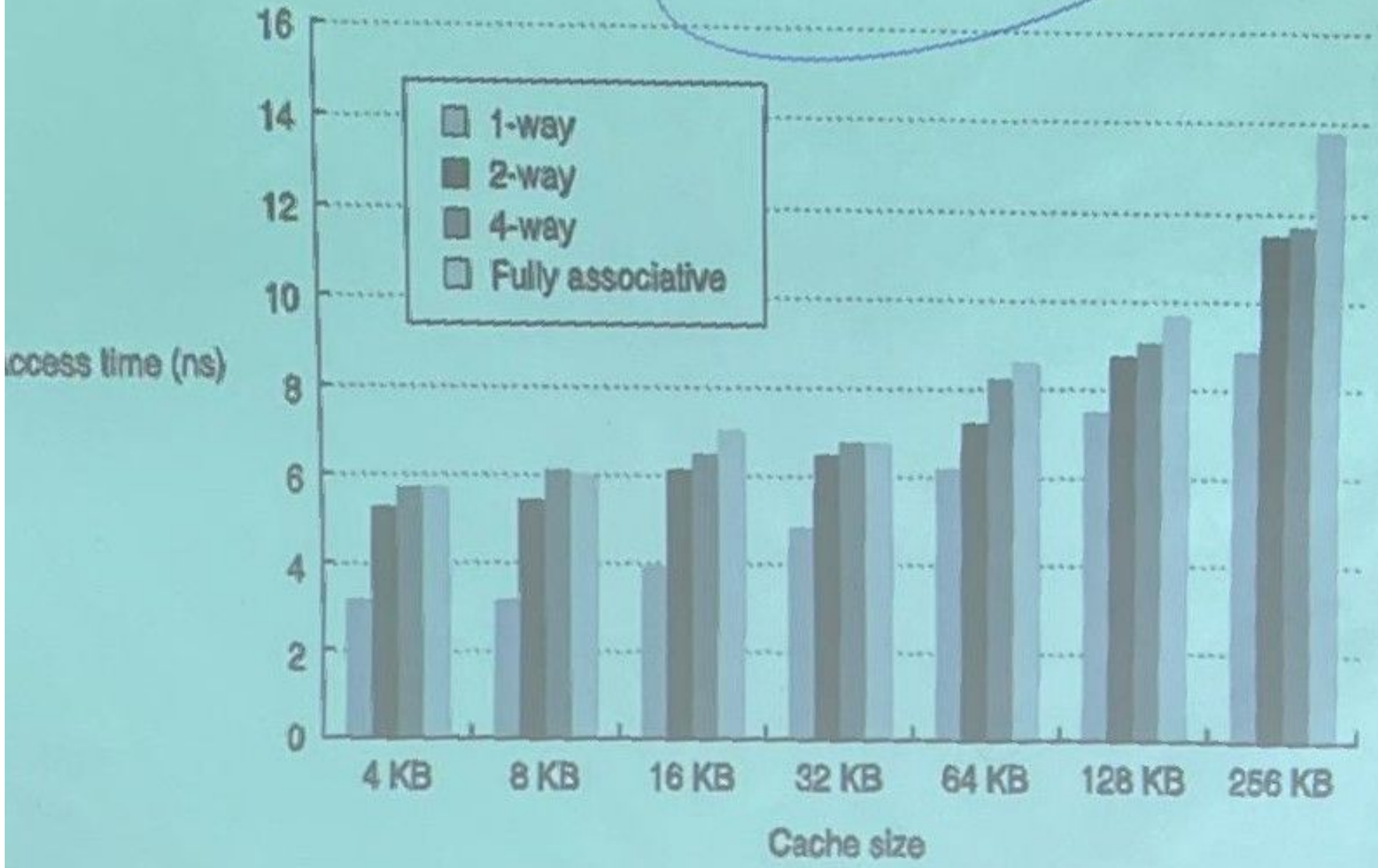
- Small cache can help hit time since smaller memory takes less time to index
  - Make cache smaller to *improve hit time*
- Or (probably better), add a new and smaller L0 cache between existing L1 cache and CPU.
- Keep L1 cache on same chip as CPU
  - Physically close to functional units that access it
- Simple cache → direct mapping
- Can overlap tag check with data transmission since no choice
- Keep L1 design simple, e.g., direct-mapped
  - Avoids multiple tag comparisons
  - Tag can be compared after data cache fetch
    - Reduces effective hit time

Access Time



Access time estimate for 90 nm using CACTI model 4.0

Access Time in a CMOS Cache

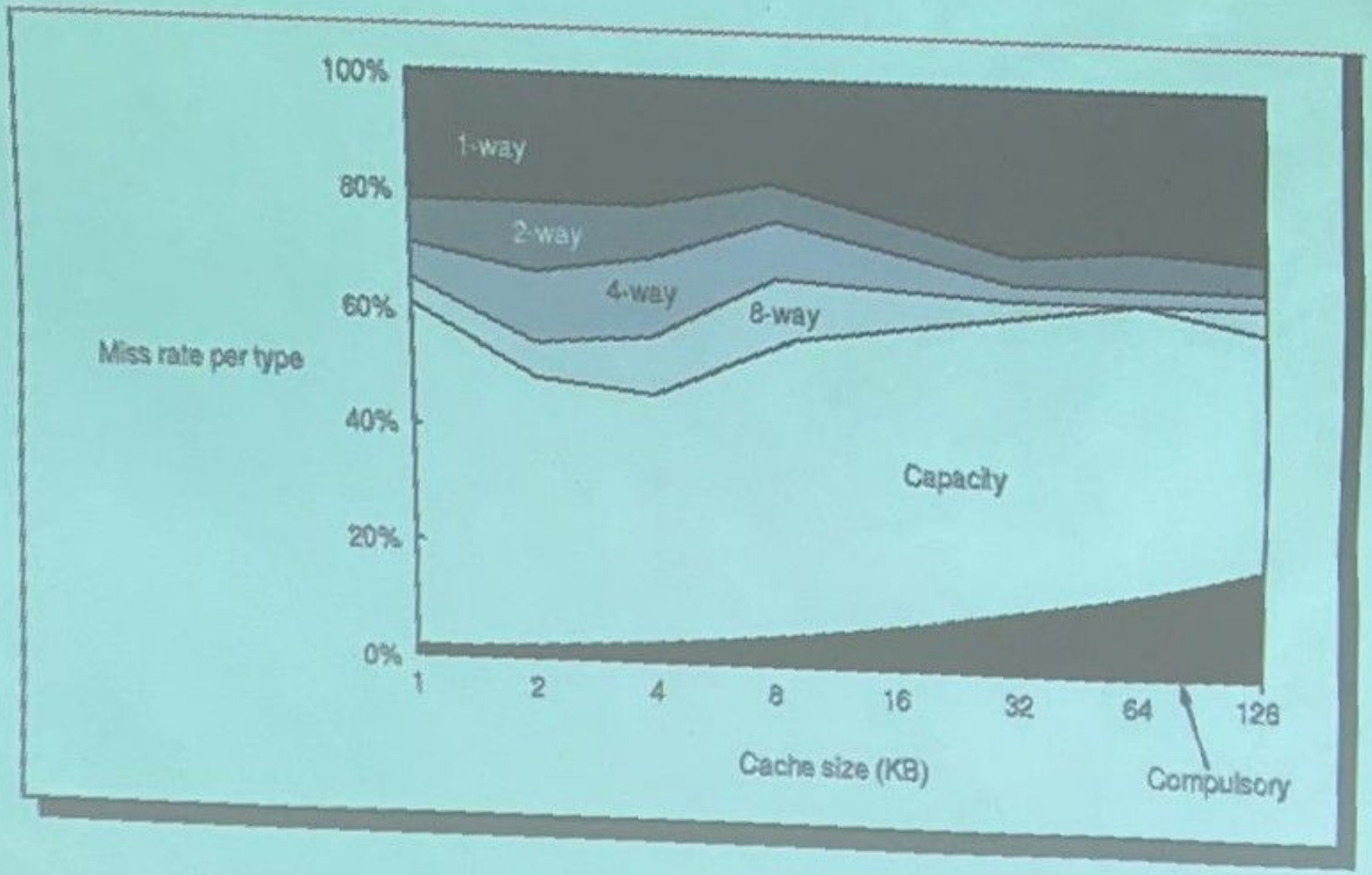


Avoid Address Translation

- In systems with virtual address spaces, virtual address must be mapped to *physical* addresses.
- If cache blocks are indexed/tagged with physical addresses, we must do this translation before we can do the cache lookup.
  - *Long hit time!*
- Solution: Access cache using the *virtual* address.
  - Call this a "Virtual Cache"
    - Drawback: Cache flush on context switch
      - Can fix by tagging blocks with Process Ids (PIDs)
    - Another problem: "***Aliasing***", i.e. two virtual addresses mapped to same real address
      - Fix with anti-aliasing or page coloring

Reducing Miss Rate

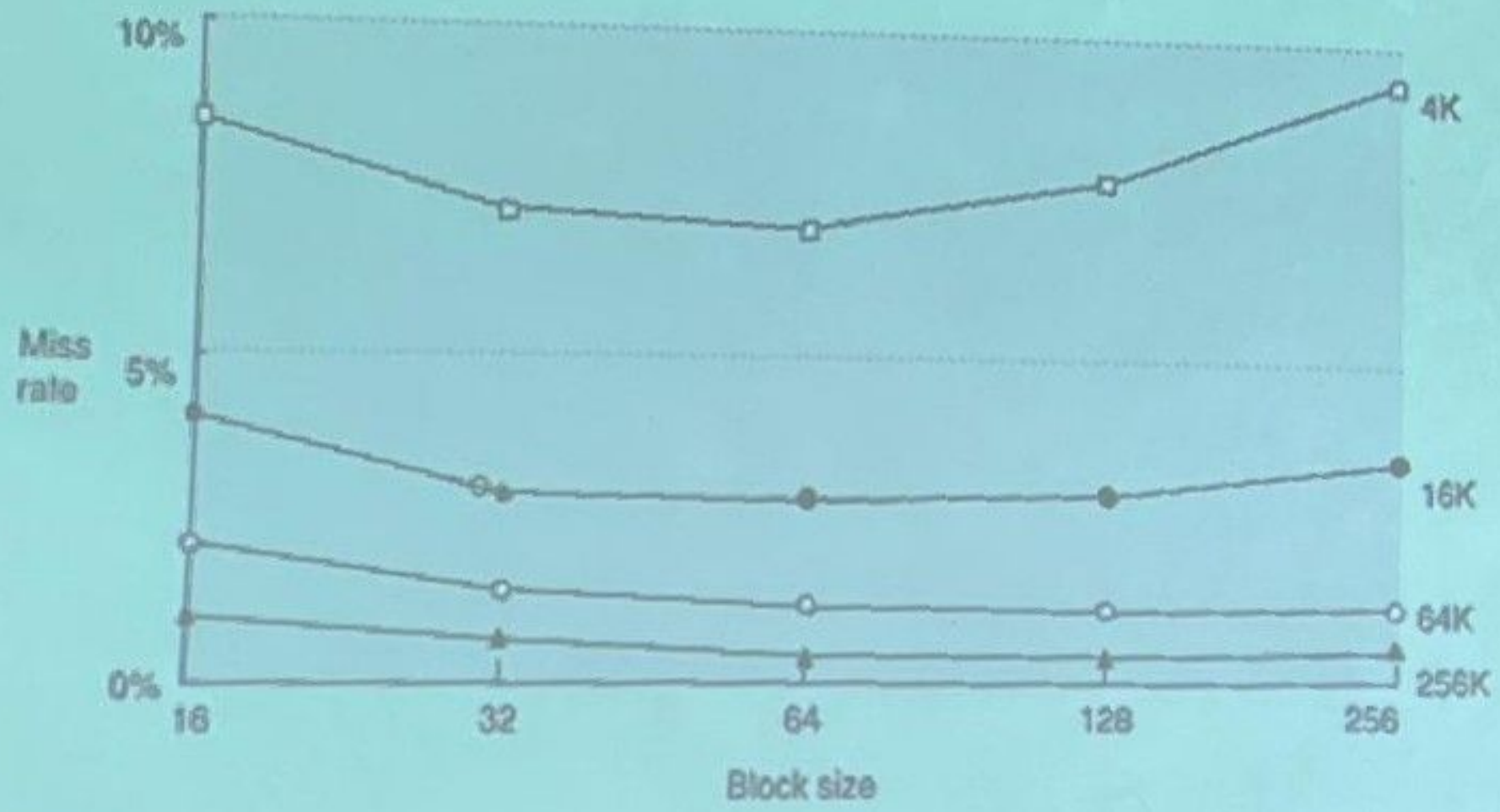
As Fraction of Total Misses



Three Types of Misses

- *Compulsory*
  - During a program, the very first access to a block will not be in the cache (unless pre-fetched)
- *Capacity*
  - The *working set* of blocks accessed by the program is too large to fit in the cache
- *Conflict*
  - Unless cache is fully associative, sometimes blocks may be evicted too early because too many frequently-accessed blocks map to the same limited set of frames.

Block Size Effect

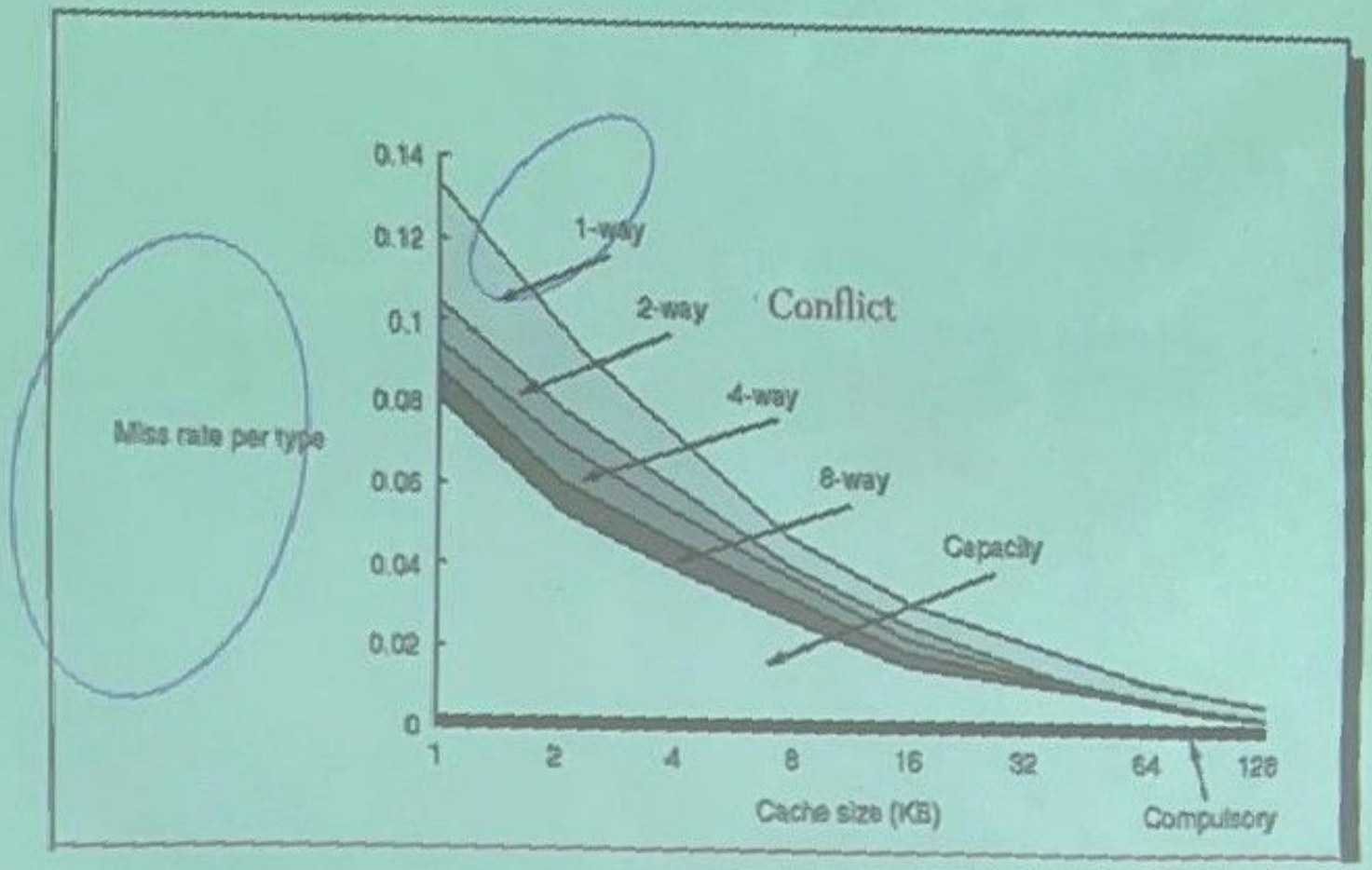


Miss rate is actually goes up if the block is too large relative to the cache size

ates For Different Cache Sizes

[illegible]	Total miss rate	Miss rate components (relative percent) (sum = 100% of total miss rate)					
		Compulsory		Capacity		Conflict	
[illegible]	0.098	0.0001	0.1%	0.070	72%	0.027	28%
[illegible]	0.076	0.0001	0.1%	0.070	93%	0.005	7%
[illegible]	0.071	0.0001	0.1%	0.070	99%	0.001	1%
[illegible]	0.071	0.0001	0.1%	0.070	100%	0.000	0%
[illegible]	0.068	0.0001	0.1%	0.044	65%	0.024	35%
[illegible]	0.049	0.0001	0.1%	0.044	90%	0.005	10%
[illegible]	0.044	0.0001	0.1%	0.044	99%	0.000	1%
[illegible]	0.044	0.0001	0.1%	0.044	100%	0.000	0%
[illegible]	0.049	0.0001	0.1%	0.040	82%	0.009	17%
[illegible]	0.041	0.0001	0.2%	0.040	98%	0.001	2%
[illegible]	0.041	0.0001	0.2%	0.040	99%	0.000	0%
[illegible]	0.041	0.0001	0.2%	0.040	100%	0.000	0%
[illegible]	0.042	0.0001	0.2%	0.037	89%	0.005	11%
[illegible]	0.038	0.0001	0.2%	0.037	99%	0.000	0%
[illegible]	0.037	0.0001	0.2%	0.037	100%	0.000	0%
[illegible]	0.037	0.0001	0.2%	0.037	100%	0.000	0%
[illegible]	0.037	0.0001	0.2%	0.028	77%	0.008	23%
[illegible]	0.031	0.0001	0.2%	0.028	91%	0.003	9%
[illegible]	0.030	0.0001	0.2%	0.028	95%	0.001	4%
[illegible]	0.029	0.0001	0.2%	0.028	97%	0.001	2%
[illegible]	0.021	0.0001	0.3%	0.019	91%	0.002	8%
[illegible]	0.019	0.0001	0.3%	0.019	100%	0.000	0%
[illegible]	0.019	0.0001	0.3%	0.019	100%	0.000	0%
[illegible]	0.019	0.0001	0.3%	0.019	100%	0.000	0%
[illegible]	0.013	0.0001	0.5%	0.012	94%	0.001	6%
[illegible]	0.012	0.0001	0.5%	0.012	99%	0.000	0%
[illegible]	0.012	0.0001	0.5%	0.012	99%	0.000	0%
[illegible]	0.012	0.0001	0.5%	0.012	99%	0.000	0%
[illegible]	[illegible]	0.0001	0.8%	0.005	86%	0.003	33%

Misses by Type



- Conflict misses are significant in a direct-mapped cache.
- From direct-mapped to 2-way helps as much as doubling cache size.
- Going from direct-mapped to 4-way is *better* than doubling cache size.

Performance Comparison

- Assume:

$$Hit\ Time_{1\text{-}way} = Cycle\ Time_{1\text{-}way} \qquad Miss\ Penalty = 25 \times Cycle\ Time_{1\text{-}way}$$
$$Hit\ Time_{2\text{-}way} = 1.36 \times Cycle\ Time_{1\text{-}way}$$
$$Hit\ Time_{4\text{-}way} = 1.44 \times Cycle\ Time_{1\text{-}way}$$
$$Hit\ Time_{8\text{-}way} = 1.52 \times Cycle\ Time_{1\text{-}way}$$

- 4KB, 1-way miss-rate=9.8%; 4-way miss-rate=7.1%

$$Average\ Mem\ Access\ Time_{1\text{-}way} = 1.00 + Miss\ Rate \times 25$$
$$= 1.00 + 0.098 \times 25 = 3.45$$

$$Average\ Mem\ Access\ Time_{4\text{-}way} = 1.44 + Miss\ Rate \times 25$$
$$= 1.44 + 0.071 \times 25 = 3.215$$

AMAT vs. Higher Set-Associativity

Cache Size	1-way	2-way	4-way	8-way
4KB	3.44	3.25	3.22	3.28
8KB	2.69	2.58	2.55	2.62
16KB	2.23	2.40	2.46	2.53
32KB	2.06	2.30	2.37	2.45
64KB	1.92	2.14	2.18	2.25
128KB	1.52	1.84	1.92	2.00
256KB	1.32	1.66	1.74	1.82
512KB	1.20	1.55	1.59	1.66

- Higher associativity increase the cycle time
- The table shows the ***average memory access time***
  - 1-way is better most of cases

Local and Global Miss Rates, Working Set

local miss rate

It is equal to the number of misses in a cache at level *i* divided by the total number of accesses at level *i*.

global miss rate

It is equal to the number of misses in a cache at level *i* divided by the total number of memory accesses.

working set

The amount of memory, a given program requires in a time interval.

AMAT for Multi-Level Cache

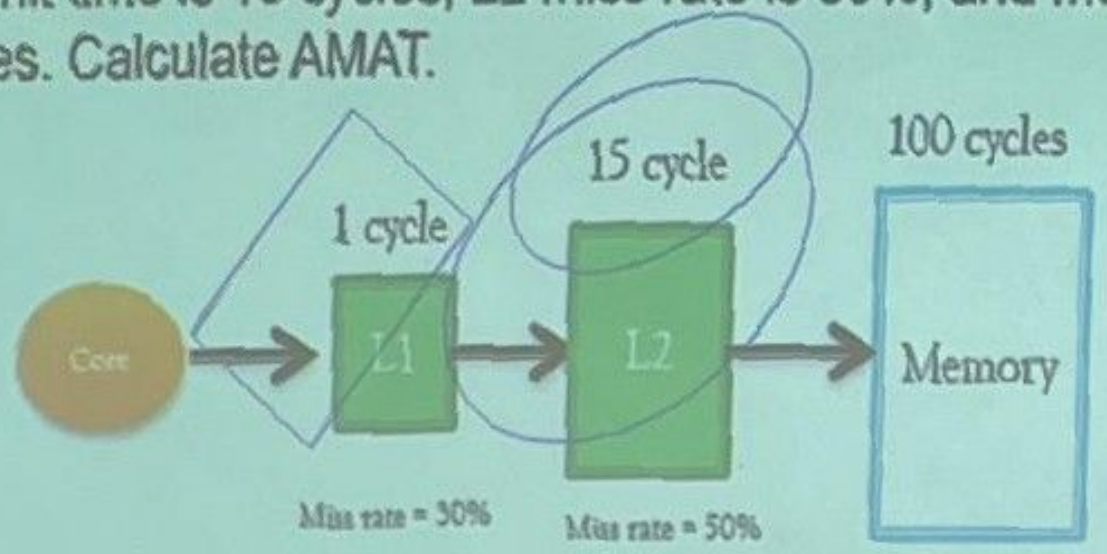
- Assuming, a memory system with an L1 and L2 cache, then

$$\begin{aligned} AMAT &= L1_{hit\ time} + L1_{miss\ rate} \times L1_{miss\ penalty} \\ &= L1_{hit\ time} + L1_{miss\ rate} \times (L2_{hit\ time} + L2_{miss\ rate} \times L2_{miss\ penalty}) \end{aligned}$$

- All the memory accesses need to access the L1 cache irrespective of a hit or a miss. Hence, they need to incur a delay equal to $L1_{hit\ time}$.
- A fraction of accesses, $L1_{miss\ rate}$, will miss in the L1 cache, and move to the L2 cache. Irrespective of a hit or a miss, we need to incur a delay of $L2_{hit\ time}$ cycles.
- If a fraction of accesses ($L2_{miss\ rate}$) miss in the L2 cache, then they need to proceed to main memory.
- We have assumed that all the accesses hit in the main memory. Hence, the $L2_{miss\ penalty}$ is equal to the main memory access time.

AMAT for Multi-Level Cache

- **Example:** Assume we have a two-level cache hierarchy system that runs an application. If you know that L1 cache hit time is 1 cycle, L1 miss rate is 30%, L2 hit time is 15 cycles, L2 miss rate is 50%, and memory latency is 100 cycles. Calculate AMAT.



- AMAT = Hit Time L1 + Miss Rate L1 X (Hit Time L2 + Miss Rate L2 X Miss penalty)
- AMAT = 1 + 30% X (15 + 50% X 100) = 20.5 cycles

n-Level Memory System

$$AMAT = L1_{hit\ time} + L1_{miss\ rate} * L1_{miss\ penalty}$$

$$L1_{miss\ penalty} = L2_{hit\ time} + L2_{miss\ rate} * L2_{miss\ penalty}$$

$$L2_{miss\ penalty} = L3_{hit\ time} + L3_{miss\ rate} * L3_{miss\ penalty}$$

$$\ldots = \ldots$$

$$L(n-1)_{miss\ penalty} = Ln_{hit\ time}$$

- Factor Influencing:
  - Local Miss Rate
  - Global Miss Rate
  - Working Set